

STAN

PHYSICAL COMPUTING 2025

RUARIDH MURDOCH ID: 02615360
COE ISHIKAWA ID: 02569394

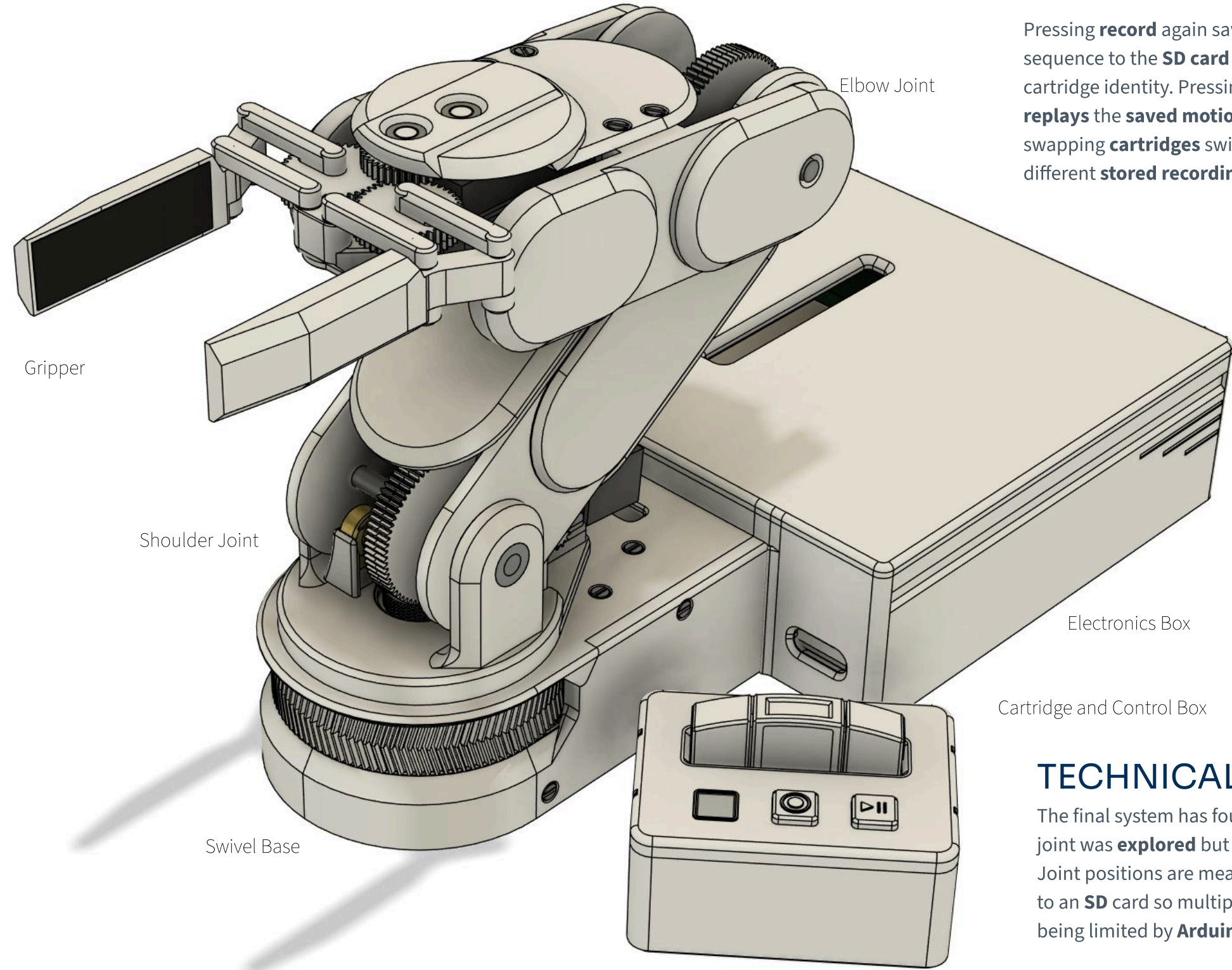


A playful, cartridge-controlled robotic arm where the user becomes the “programmer” by physically moving the joints. Insert a coloured cartridge to select a slot, press record, and pose the arm through any motion by hand; the joint angles are captured and saved. Press playback to repeat the movement automatically, and swap cartridges to store and replay different sequences like a set of physical motion “games.”

INTERACTION DESIGN

CONCEPT DEVELOPMENT

The brief asked for a “**resourceful**” Gizmo that achieves **interesting** behaviour using **limited, low-cost mechatronic** elements. We initially considered high-torque servos, but moved away from this because it relies on expensive components rather than **mechanical design**. Through ideation we developed a robotic arm that can be physically **moved, recorded, and replayed**, aiming for a fun **toy-like** interaction while still being **mechanically robust**.



Resourcefulness is shown by effectively building “**servo-like**” joints from **standard DC motors**, gear reductions, and **potentiometers**, rather than buying integrated actuators. The same hardware can also produce many behaviours because motion is defined in software and stored as recordings. We produced all the **CAD and code** ourselves, and most structural parts are **3D printed**, with bearings and shafts used where **strength** and alignment are critical.

INTERACTIVITY

The user inserts a **coloured cartridge** to select an active slot. Pressing **record** starts capture while the arm is moved by hand, with **joint** angles **logged** over time from the **potentiometers**.

Pressing **record** again saves the sequence to the **SD card** under that cartridge identity. Pressing playback **replays** the **saved motion**, and swapping **cartridges** switches between different **stored recordings**.

SET CARTRIDGE



RECORD



PLAYBACK

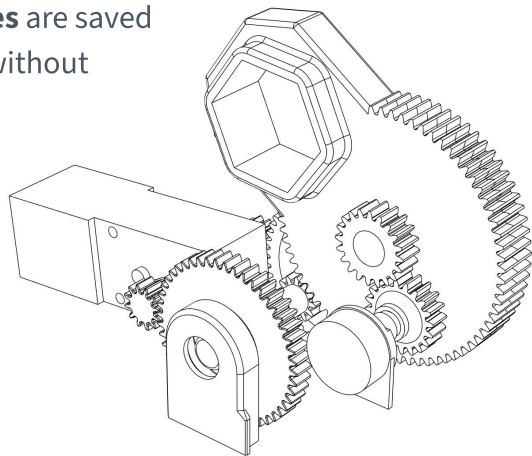
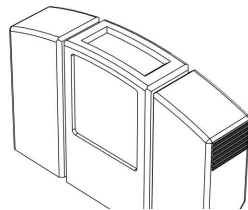


Cartridge and Control Box

TECHNICAL OVERVIEW

The final system has four joints: **swivel, shoulder, elbow, and gripper**. A wrist joint was **explored** but removed to reduce **space, weight, and complexity**. Joint positions are measured with **potentiometers** and **sequences** are saved to an **SD card** so multiple recordings can be stored and replayed without being limited by **Arduino memory**.

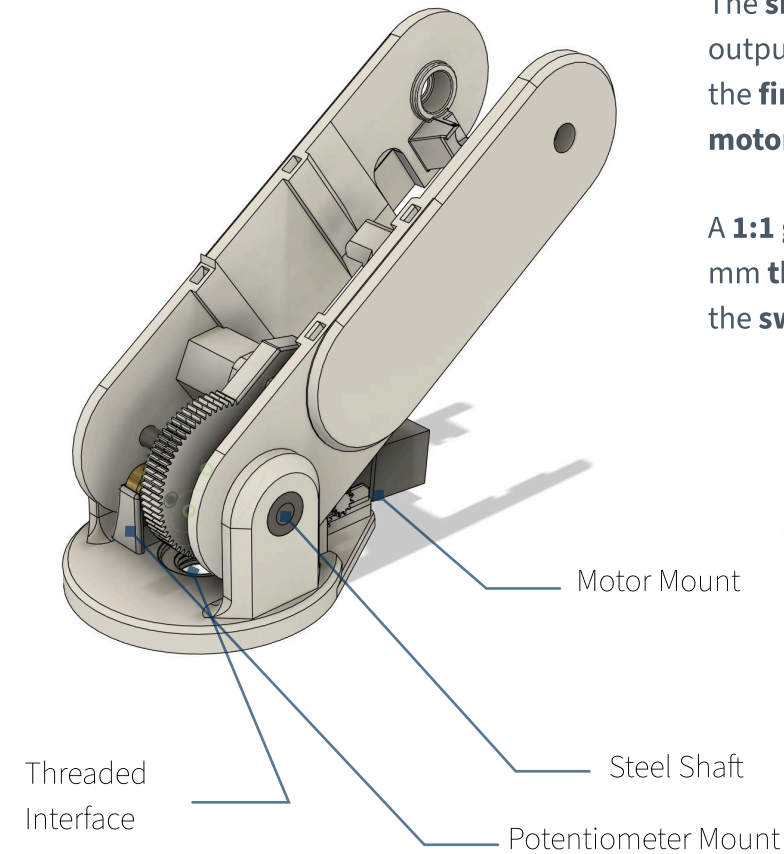
We originally planned **NFC-based** selection, but switched to a colour-based cartridge system because it is **easier to manufacture** in unlimited quantities, **simpler** to calibrate, and gives a clearer **physical interaction**.



MECHANICAL DESIGN

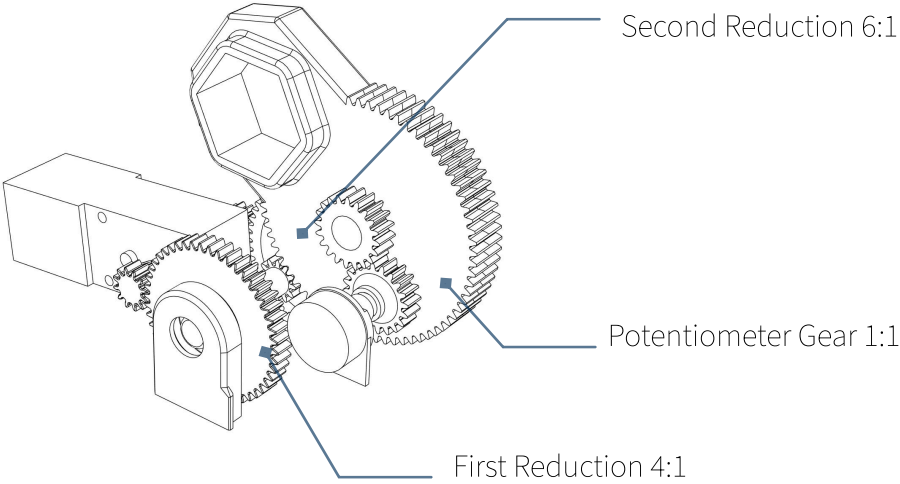
The joint system was iteratively designed to deliver robust, back drivable motion while being able to be assembled and disassembled easily for maintenance.

SHOULDER JOINT



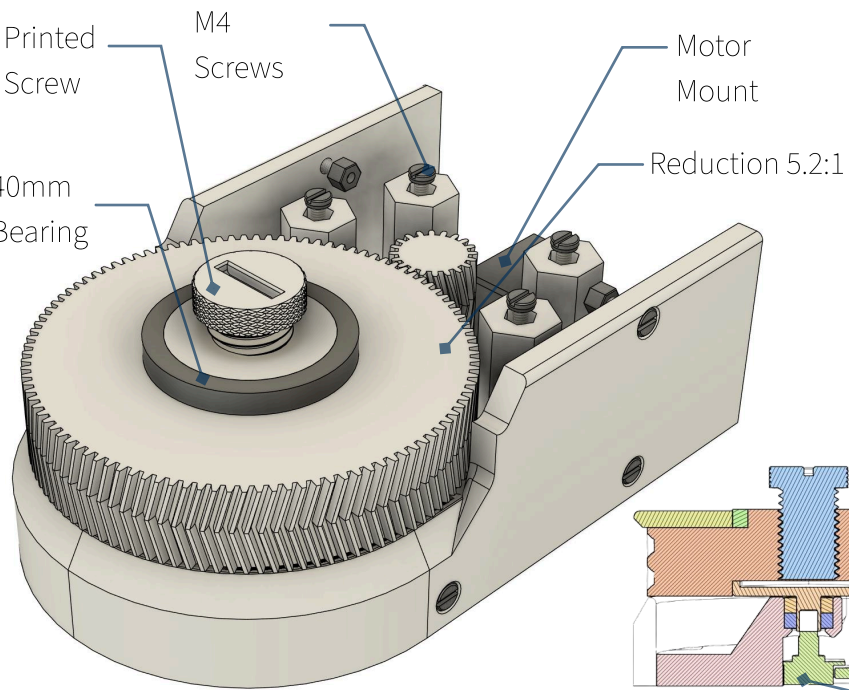
The **shoulder joint** drives lift through a **spur gear reduction** that turns the large output gear attached to the upper arm. The *first build* used **12:1 reduction**, and the **final build** used **24:1** (48:12, then 72:12). The geartrain is driven from **both motor output shafts** to **split contact forces** across both sides and reduce wear.

A **1:1 gear pair** links the **output gear to the potentiometer**, and a central 20 mm **threaded** hole allows an insert to be placed that **clamps the shoulder** to the **swivel joint** to maintain a **fixed orientation**.



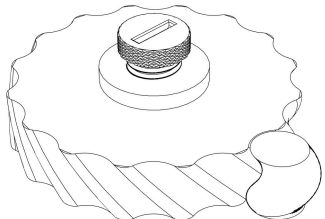
Testing showed the 12:1 (30:15, then 72:12) setup **stalled** under the target load, so the ratio was **increased** to 24:1 to **raise torque** and improve control. Backdriving increased tooth loading, so the **gears were reinforced** with higher infill and chamfers to reduce tooth damage during repeated use.

SWIVEL BASE



The **swivel joint** provides **base rotation** using a 5.2:1 **herringbone gear set** (104:20). The large gear is supported by 8 mm lower bearings and a 40 mm upper bearing for alignment, with the **potentiometer mounted underneath**.

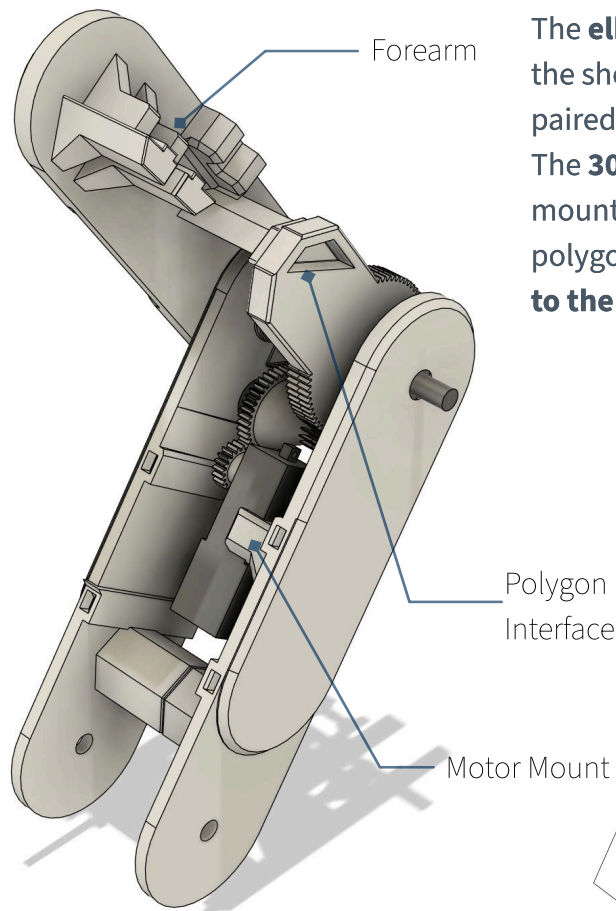
The housing is assembled with **M4 split screws** into glued-in **M4 Nylock nuts**, with **rear attachment** points for add-ons.



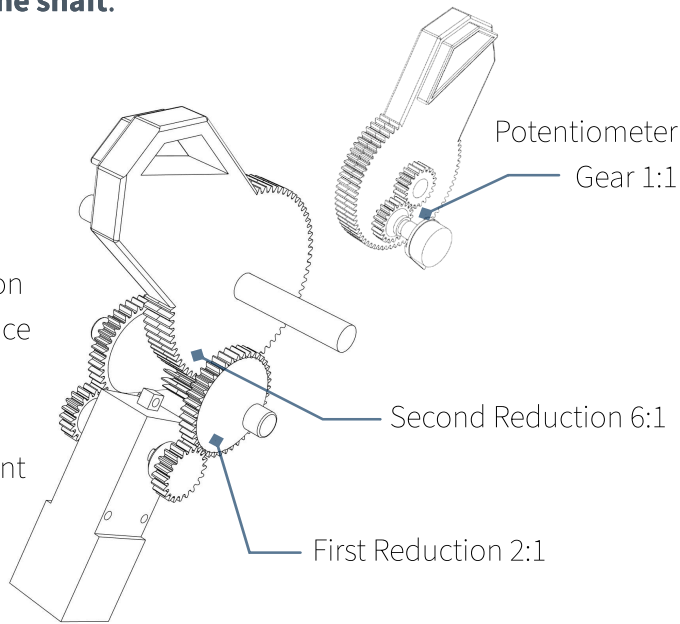
Early versions used a **16:1 cycloidal drive** that slipped under backdriving, so it was replaced with the herringbone set, **keeping the same** centre **distances** to minimise reprints.

A threaded hole in the large gear accepts the **printed screw**

ELBOW JOINT

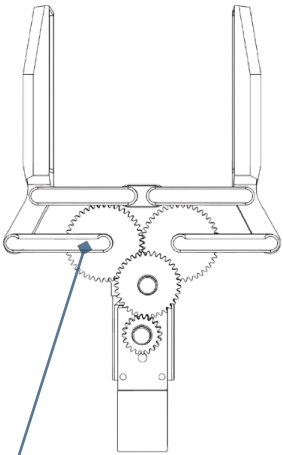


The **elbow joint** uses the same **spur-gear** layout as the shoulder with a **12:1 reduction** (72:12 with paired 30:15 gears) to rotate the forearm output gear. The **30T** gear sits on a **removable** bearing-supported mount, the joint runs on an 8 mm steel shaft, and a polygon interface **transmits torque without gluing to the shaft**.



The elbow kept 12:1 because it **met the load** and motion **requirements without stalling**, so no higher reduction was needed. The removable mount and polygon interface were included to make **adjustments** and **repairs** during iteration.

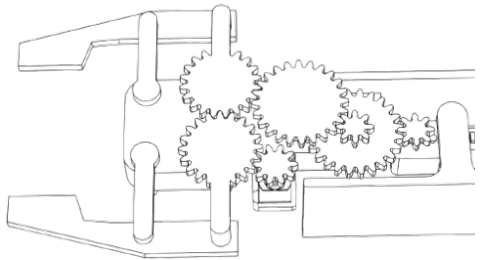
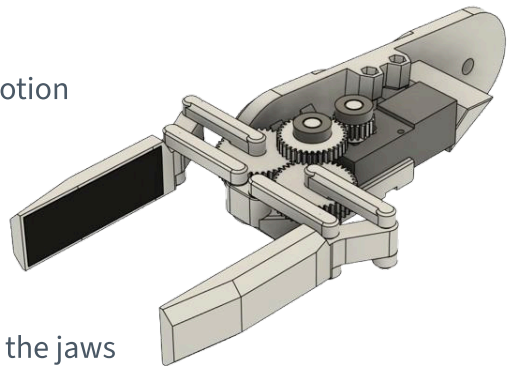
GRIPPER MECHANISM



The **gripper** creates a closing motion using a **4-bar parallel linkage**, keeping both jaws parallel as they move.

The final build uses a **6:1 gear reduction**, with rubber pads on the jaws to improve grip.

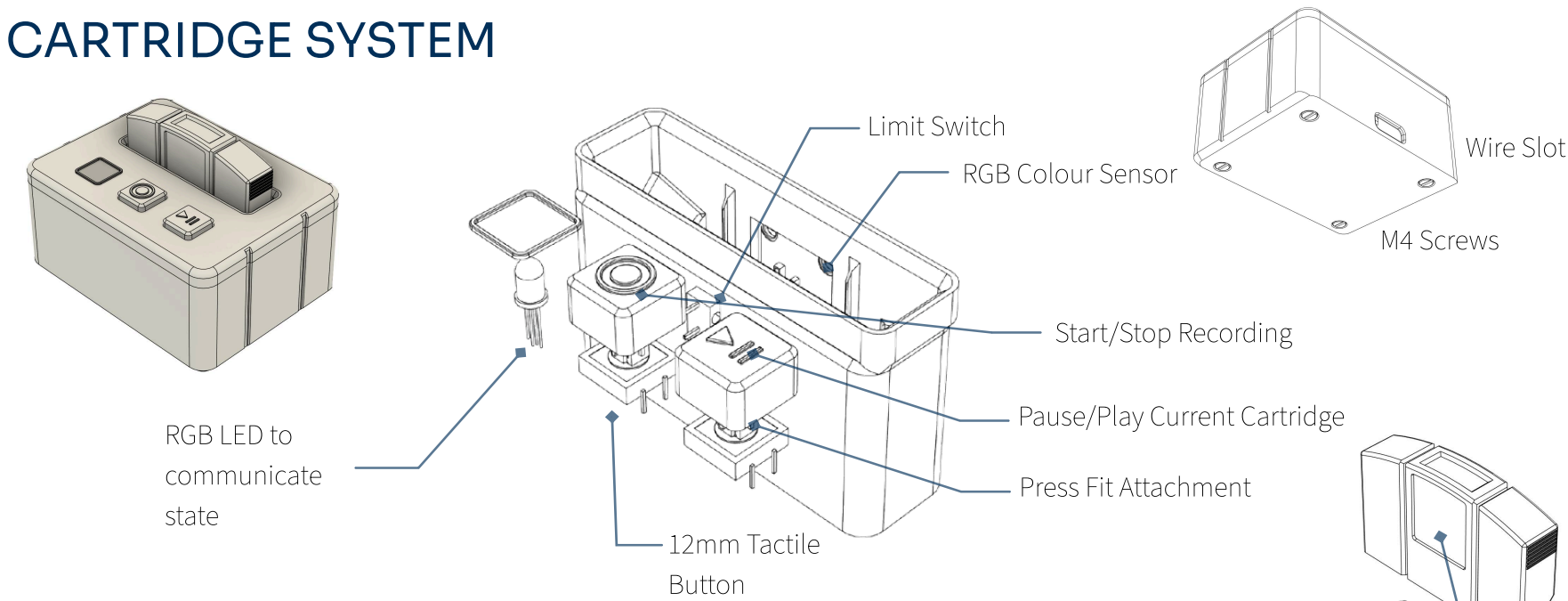
The whole mechanism mounts to the forearm on an **angled bracket** and can be removed using the same **M4 screw** fixing system.



Early versions suffered from gear slip and tilt that caused **locking**, and the mechanism was **oversized** and **unintuitive**, making grasp placement hard. The redesign improved **alignment** and **reliability** by using **bearings** to hold the gears upright.

ELECTRONICS DESIGN

CARTRIDGE SYSTEM

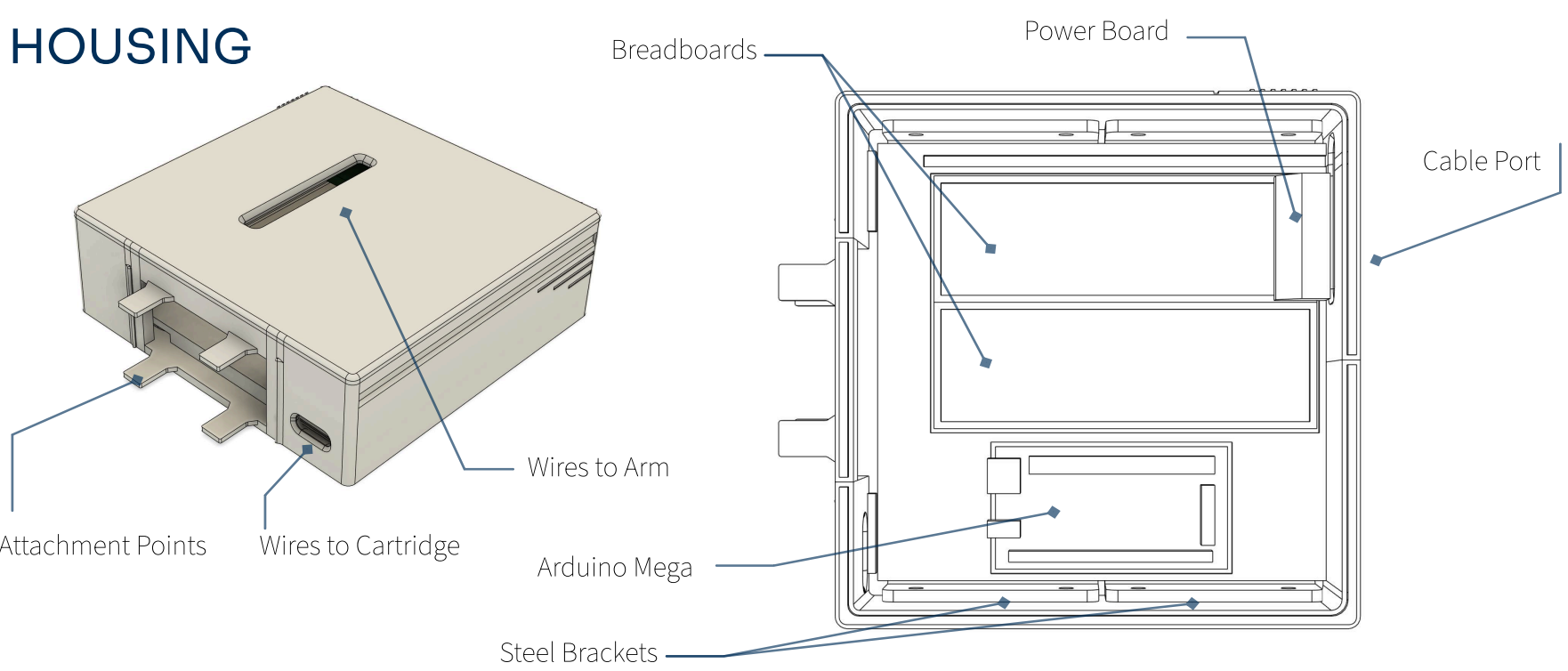


The **cartridge system** is an **external** module that **controls recording** and **playback** through a simple insert-and-press interaction. A **limit switch** confirms cartridge insertion, and an **RGB colour sensor** reads a **coloured** patch on the cartridge to **identify** which sequence to load or save.

The cartridge was styled like a **N64 game cartridge**, and the keyed **slot** guides it in smoothly with designed clearances so it **only inserts one way**.

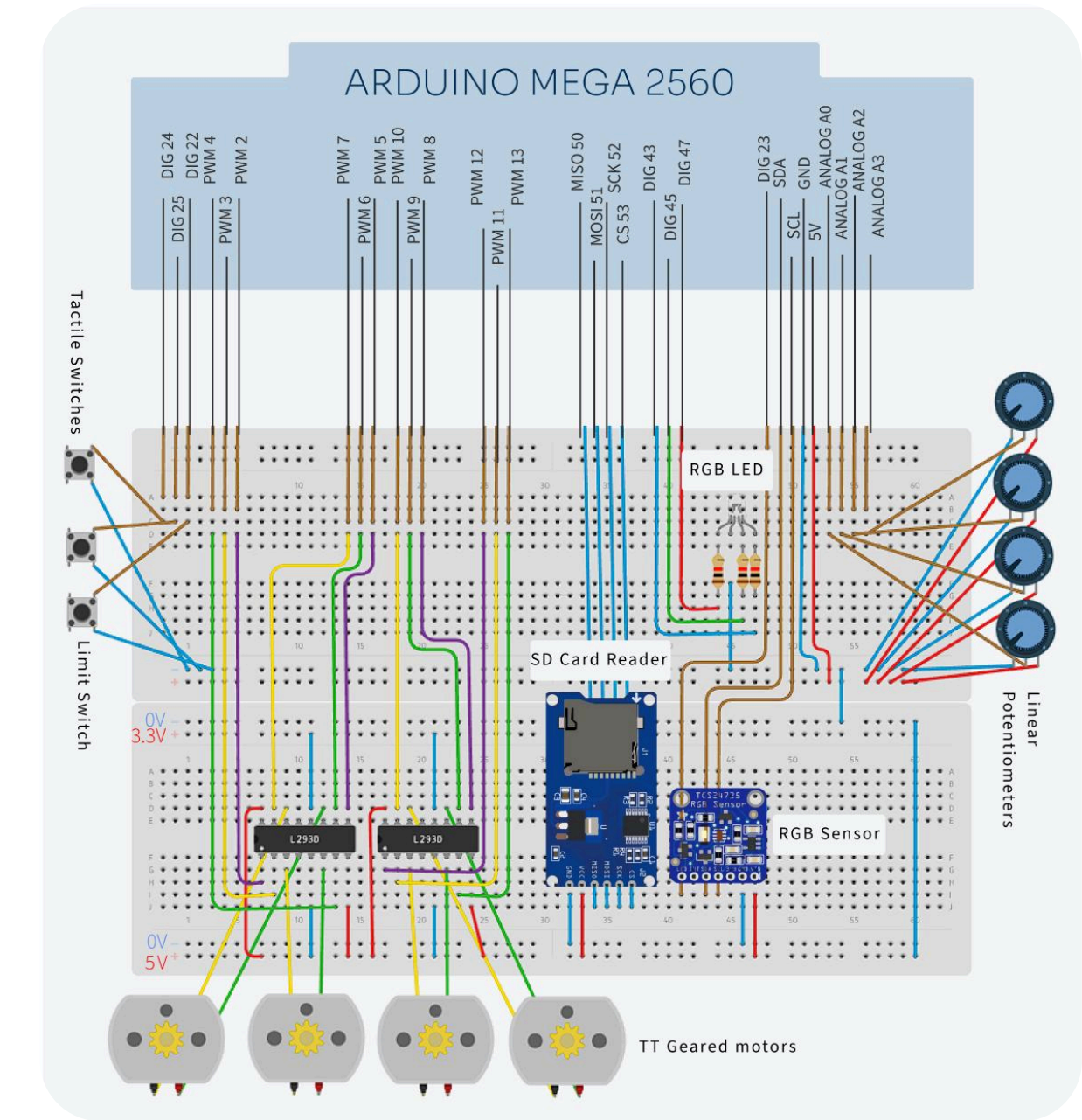
Used an **SD card** to store recorded motion because the **Arduino memory cannot hold multiple long recordings**. Each recording is written as a file **linked** to the **cartridge colour**, and the data is non-volatile, meaning recordings **remain available** after the system is powered off and can be **replayed later**.

HOUSING



Houses the **Arduino Mega**, two **breadboards**, and **wiring** in a protected base. It has a top cable slot, a side cut-out for the cartridge module, and rear access for power and the **Arduino** cable. The **power bank** stays **external**, and the base is weighted with **four 300 g salvaged steel brackets** to prevent tipping.

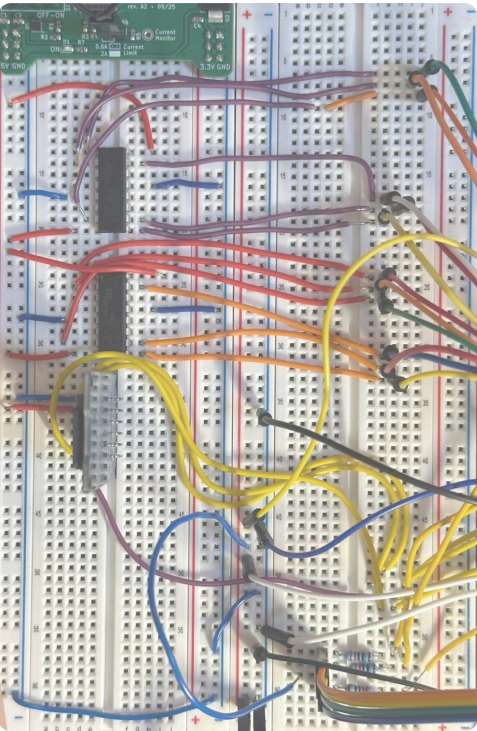
CIRCUIT DIAGRAM



Linear Potentiometers:
Used 10 kΩ linear potentiometers for joint position feedback. The Arduino reads 0–1023 and maps this to the joint angle. Chosen for simple, repeatable sensing. It was powered from a separate rail to reduce motor-noise interference.

TT motors:
Used geared TT motors for higher output torque than cheap DC motors, reducing how much extra gear reduction was needed in printed parts.

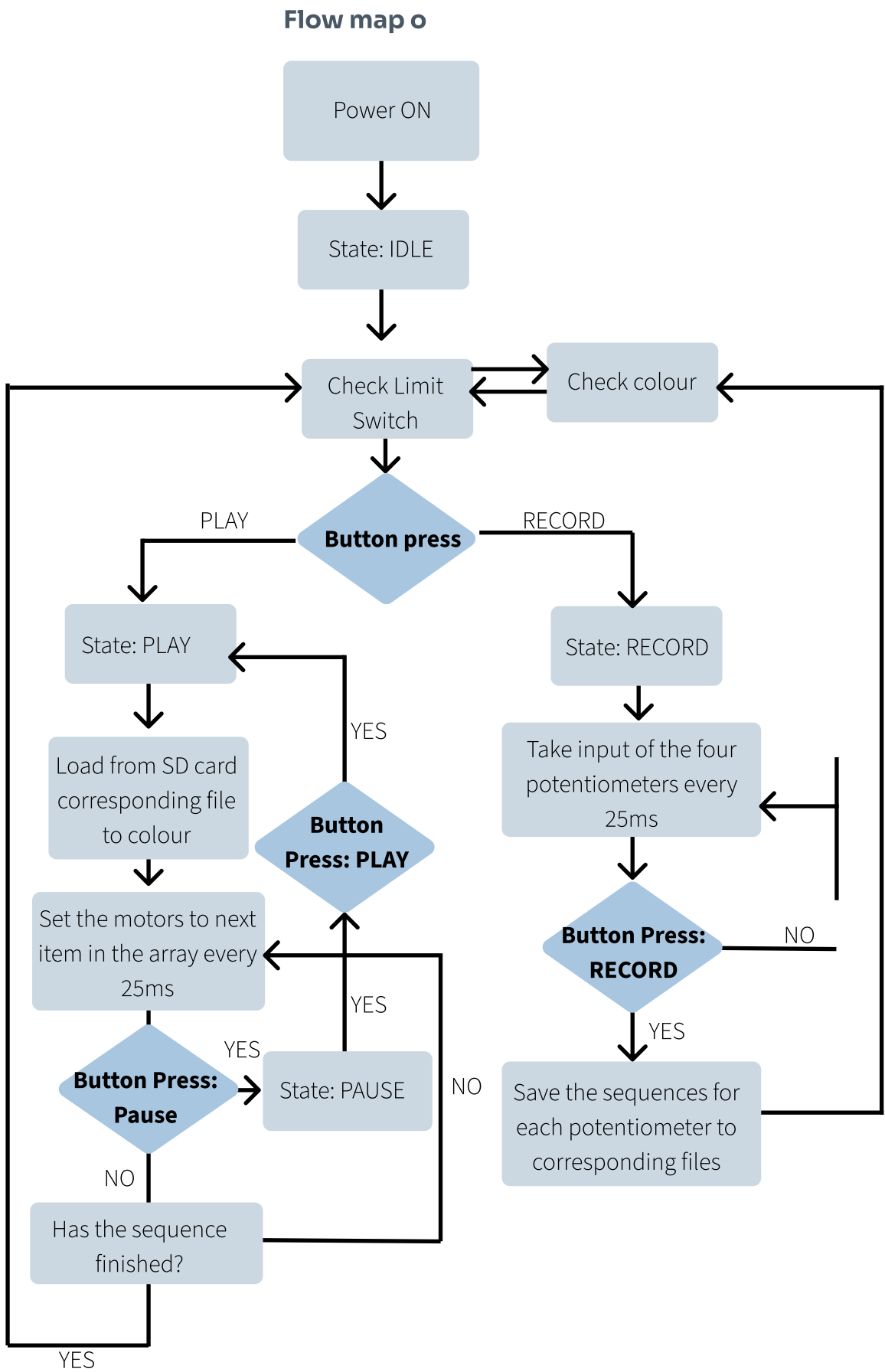
L293D:
Used L293D H-bridges to drive the DC motors and control rotation direction from the Arduino.



SOFTWARE DESIGN

FLOW MAP

Flow map of software and user interaction



SOFTWARE DESIGN

PLAYBACK LOGIC

Playback uses **time-based synchronisation** rather than simply iterating through frames sequentially. The **elapsed time** since playback started is compared against stored **timestamps** to find the correct frame, which ensures the motion is reproduced at the **original speed** regardless of loop execution time. **Pausing** is handled by accumulating **paused duration** and subtracting it from the elapsed time calculation.

MOTOR CONTROL

Motor control uses (on-off) control with **tolerance**. Each motor is driven at full speed towards its **target position** until it falls within the tolerance threshold, at which point it stops. This is simple but effective for **position matching** without requiring **PID** tuning.

STATE MACHINE

The system operates using a finite state machine with four states: idle, recording, playback, and paused. This approach keeps the main loop simple and allows clean transitions between modes based on user input.

RECORDING

During recording, **potentiometer** values are sampled at **25ms intervals** and stored in parallel arrays along with timestamps. Using separate arrays for each motor's position data rather than an array of **structs** reduces memory overhead on the microcontroller. The system can store up to **800** frames, giving approximately **20** seconds of recording time.

COLOUR DETECTION

The colour detection system maps **RGB** sensor readings to predefined calibrated values for each **cartridge colour**. When a match is found within tolerance, the corresponding filename is used for saving and loading recordings. This allows **multiple recordings** to be stored and recalled by swapping **physical cartridges**.

File I/O uses a simple **CSV format** with the **frame** count on the first line followed by comma-separated values for each frame. This makes the data **human-readable** and **easy to debug** if needed.

IMPLEMENTATION

```
// Calculate elapsed time (excluding paused time)
unsigned long elapsed = millis() - playStart - pauseTotal;
```

```
if (abs(diff) <= MOTOR_TOLERANCE) {
    // Close enough, stop motor
    digitalWrite(en, LOW);
} else {
    // Drive motor in correct direction
    digitalWrite(en, HIGH);
    if (diff > 0) {
        digitalWrite(a, LOW);
        digitalWrite(b, HIGH);
    } else {
        digitalWrite(a, HIGH);
        digitalWrite(b, LOW);
    }
}
```

```
63 // State machine states
64 #define ST_IDLE 0
65 #define ST_REC 1
66 #define ST_PLAY 2
67 #define ST_PAUSE 3
68
```

```
unsigned long now = millis();

// Only record every RECORD_MS milliseconds
if ((now - lastRec) < RECORD_MS) return;
lastRec = now;
```

```
// Write each frame as CSV
for (int i = 0; i < frameCount; i++) {
    f.print(frameTime[i]);
    f.print(',');
    f.print(frameM1[i]);
    f.print(',');
    f.print(frameM2[i]);
    f.print(',');
    f.print(frameM3[i]);
    f.print(',');
    f.println(frameM4[i]);
}
```